



UNLOX

Containerization & Orchestration for ML Systems

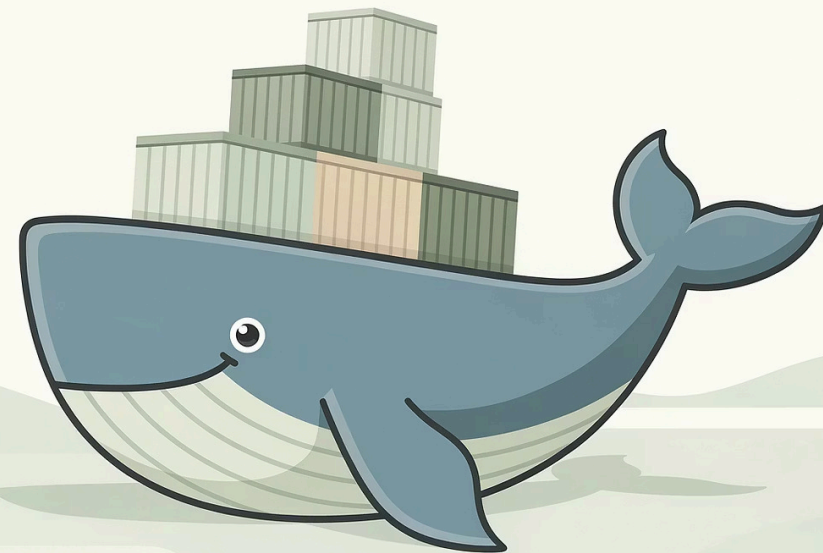
A practical guide to deploying and scaling machine learning models with **Docker** and **Kubernetes** — the foundational infrastructure of modern MLOps.

UNLOX

CHAPTER 27.1

What Is Docker?

Docker is an open-source platform for building, sharing, and running applications inside isolated environments called **containers**. A container is a lightweight, standalone, executable package that bundles everything required to run a piece of software — the application code, a language runtime, system libraries, and configuration files. Unlike virtual machines, containers share the host OS kernel, making them dramatically faster to start and far more resource-efficient. Docker has become the de facto standard for packaging software across development, testing, and production environments.



UNLOX

The Problem Docker Solves

"It Works on My Machine"

Every data scientist has heard this phrase. A model trains perfectly in a Jupyter notebook on your laptop, but crashes when a colleague tries to reproduce it — or when it reaches production. Subtle differences in Python versions, CUDA drivers, library builds, or OS-level dependencies silently break things.

How Containers Fix This

Docker eliminates environment drift by capturing **the entire runtime context** in a portable image. That image runs identically on a developer's Mac, a CI/CD server, and a Kubernetes cluster in the cloud. For MLOps, this means your trained model, its inference code, and every dependency travel together as a single immutable artifact — deployable anywhere, every time.

Anatomy of a Docker Container

Understanding what goes into a container helps you build lean, secure images for your ML workloads.



Application Code

Your inference script, API server, or pipeline logic — the core software that performs predictions.



Runtime & Libraries

Python, PyTorch, TensorFlow, scikit-learn — pinned to exact versions so nothing drifts.



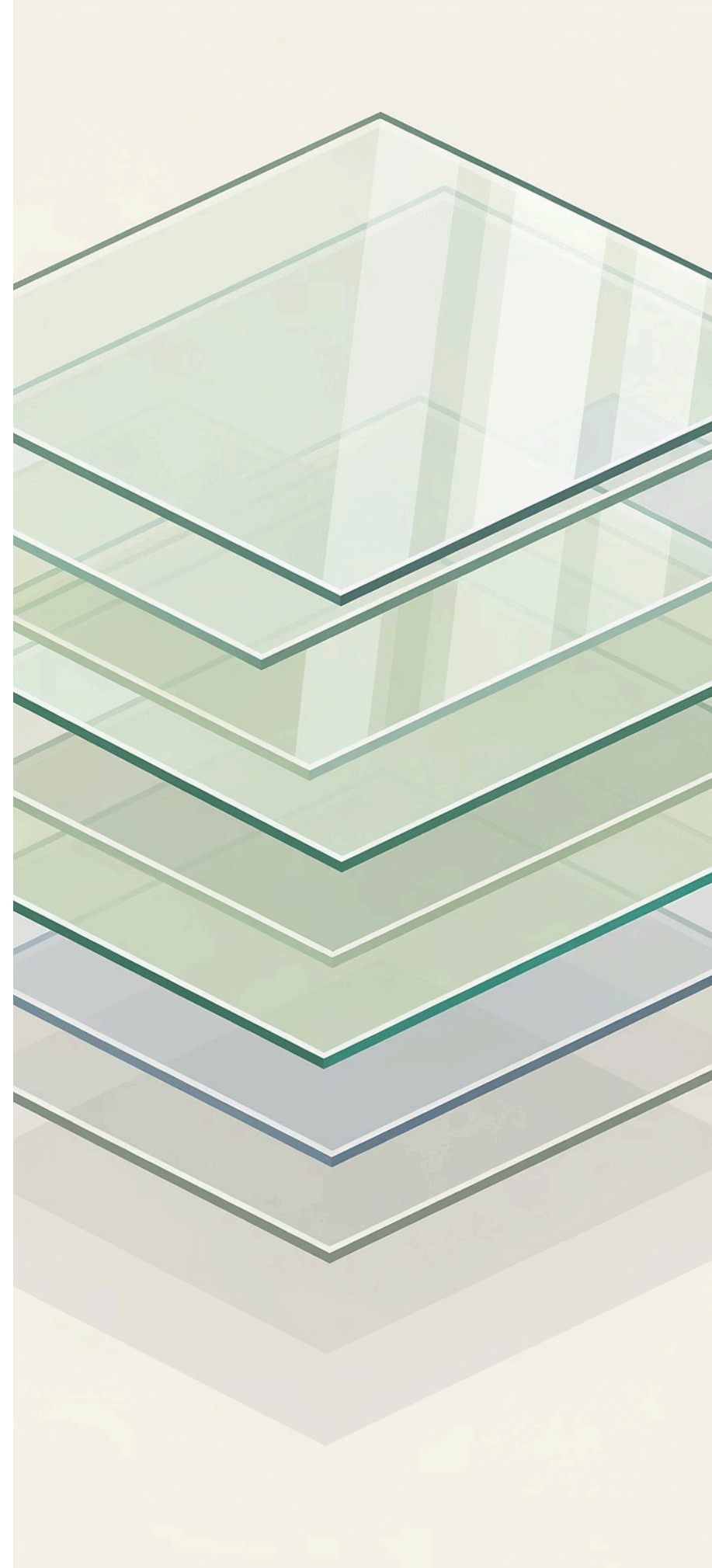
System Tools

OS-level packages, CUDA toolkit, and utilities required by your dependencies.

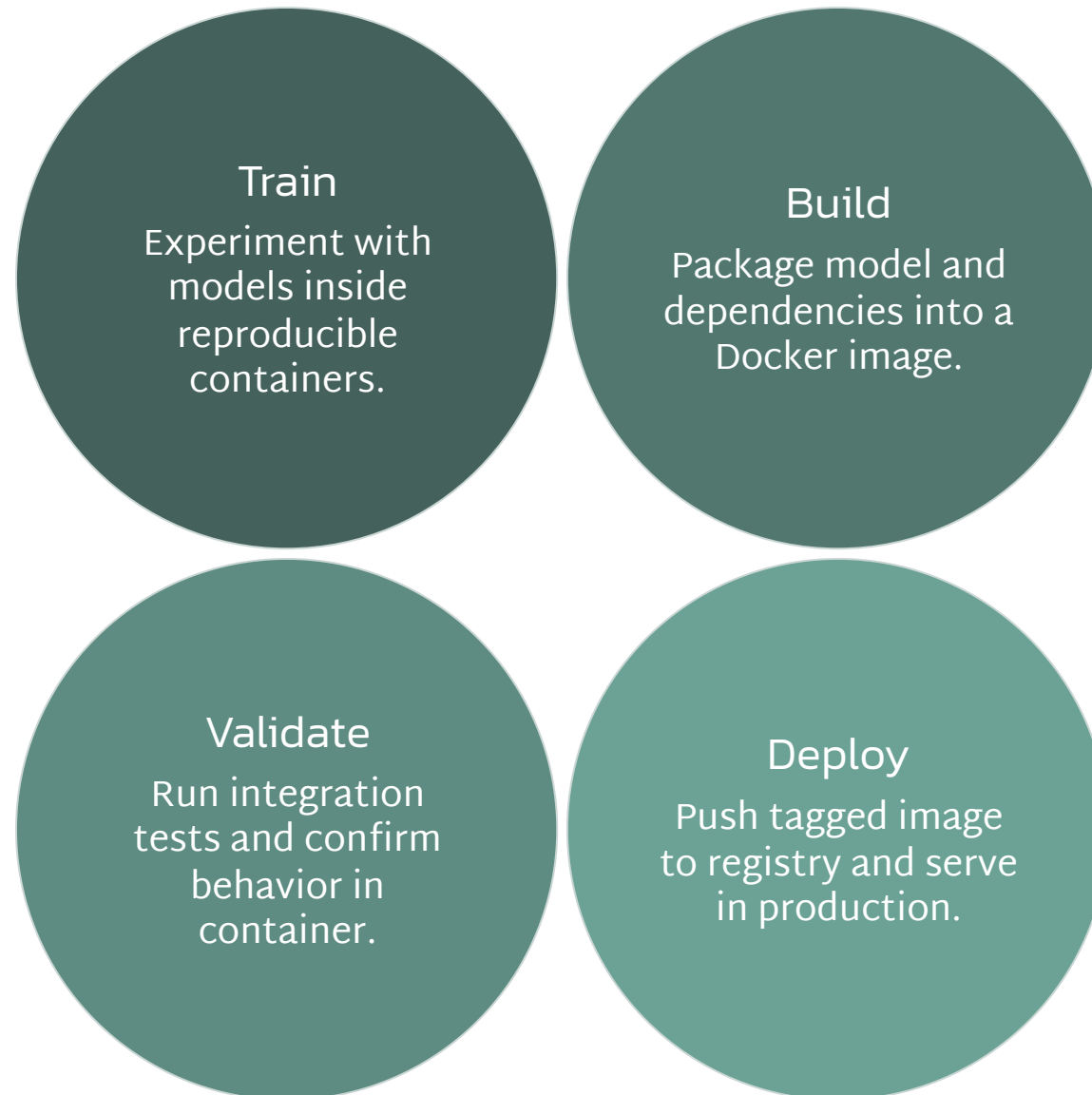


Configuration

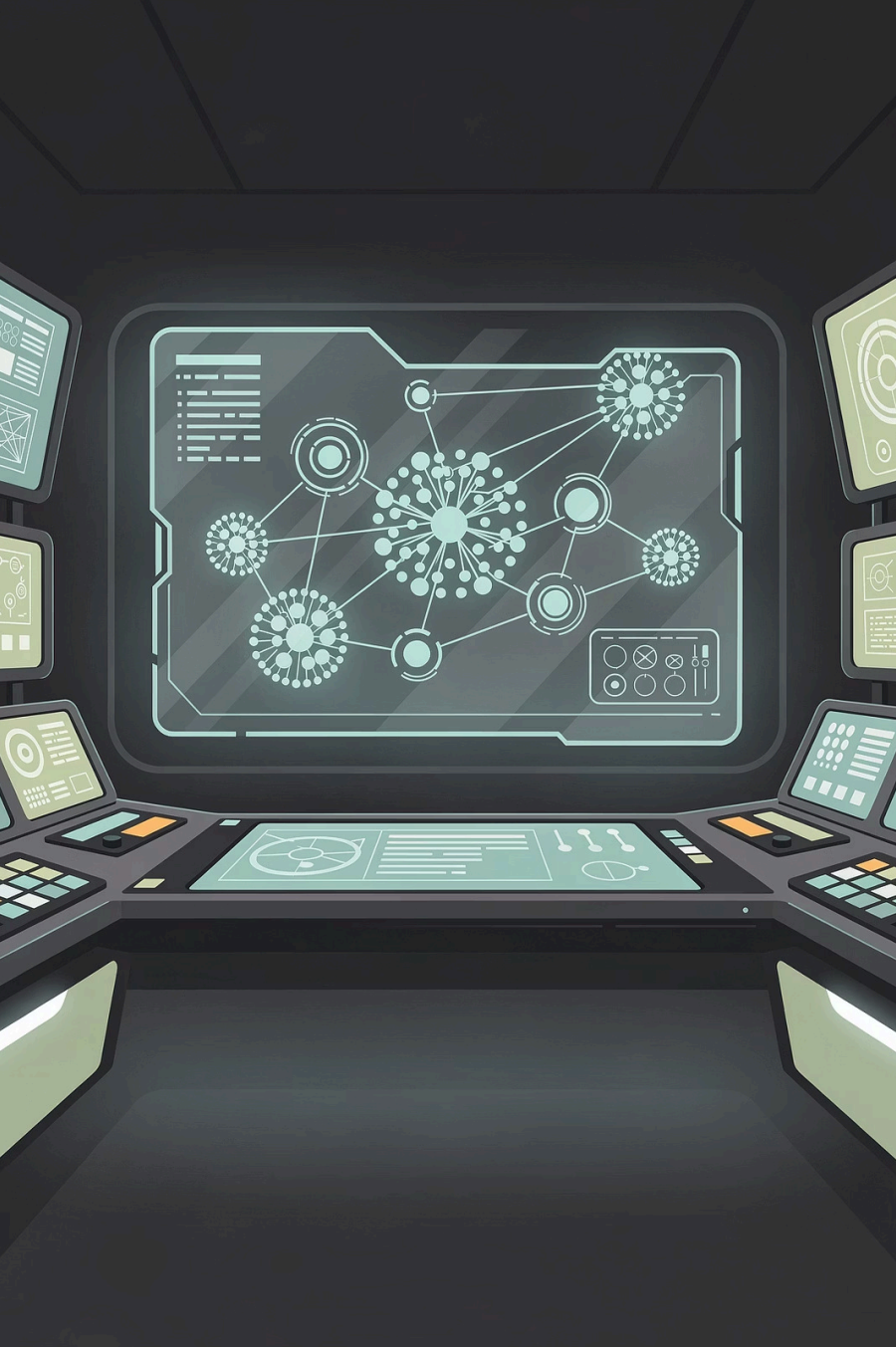
Environment variables, config files, and model artifacts bundled for reproducibility.



Docker in the ML Lifecycle



By containerizing every stage — from experimentation to production serving — teams gain **reproducibility**, **portability**, and **version control** over their entire ML pipeline. Each image is tagged and stored in a registry, creating an auditable history of every model version ever deployed.



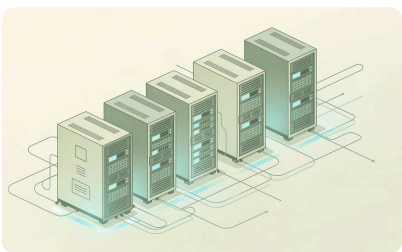
UNLOX

CHAPTER 27.2

Enter Kubernetes

Docker excels at running a *single* container, but production ML systems rarely consist of just one. You need model servers, feature stores, monitoring agents, and API gateways — all running, scaling, and healing independently. **Kubernetes** (often abbreviated **K8s**) is an open-source orchestration platform originally designed by Google that automates the deployment, scaling, and management of containerized applications across clusters of machines. It has become the backbone of cloud-native MLOps infrastructure.

Core Kubernetes Capabilities



Cluster Management

Kubernetes organizes a pool of physical or virtual machines into a unified **cluster**. It schedules containers onto nodes based on resource availability, affinity rules, and GPU requirements — critical when serving GPU-intensive deep learning models.



Load Balancing

Built-in service discovery and load balancing distribute incoming inference requests evenly across healthy replicas. This ensures your model API can absorb traffic spikes — from hundreds to millions of predictions per second — without manual intervention.



Helm Charts

Helm is the **package manager for Kubernetes**. It bundles complex multi-container deployments into reusable, versioned *charts* — defining services, config maps, secrets, and scaling policies in a single declarative template you can install with one command.

Scaling ML Models with Kubernetes

One of the most powerful features for MLOps engineers is Kubernetes' ability to scale workloads automatically based on real-time demand.

1

Horizontal Pod Autoscaler

Monitors CPU, memory, or custom metrics (like inference latency) and automatically adds or removes pod replicas. When traffic surges, new model-serving pods spin up in seconds.

2

Rolling Deployments

Deploy a new model version with zero downtime. Kubernetes gradually replaces old pods with new ones, rolling back automatically if health checks fail — enabling safe canary and blue-green releases.

3

Resource Quotas & Limits

Assign GPU and memory limits per workload so that a runaway training job never starves your production inference service. Namespaces isolate teams and environments within the same cluster.

4

Self-Healing

If a container crashes, Kubernetes restarts it automatically. If a node fails, pods are rescheduled to healthy nodes — keeping your serving infrastructure resilient around the clock.

Docker vs. Kubernetes at a Glance

Dimension	Docker	Kubernetes
Primary Role	Build & run individual containers	Orchestrate fleets of containers
Scaling	Manual (docker-compose scale)	Automatic horizontal & vertical scaling
Networking	Simple bridge / host networking	Service mesh, ingress controllers, DNS
Self-Healing	Restart policies only	Rescheduling, liveness & readiness probes
Package Mgmt	Dockerfile / Docker Compose	Helm Charts for complex deployments
Best For	Local dev, CI/CD builds, single-node	Production, multi-node, high availability

Docker provides the **packaging** layer; Kubernetes provides the **operations** layer. Together, they form the infrastructure backbone of scalable MLOps.



UNLOX

Putting It All Together

A production-grade ML deployment pipeline leverages both Docker and Kubernetes in concert. Here's the typical end-to-end flow that MLOps teams follow:



Containerize

Write a Dockerfile that installs dependencies and copies model artifacts into an image.



Push to Registry

Tag and push the image to a container registry (ECR, GCR, Docker Hub).



Deploy via Helm

Use a Helm chart to define replicas, resources, and health checks, then deploy to the K8s cluster.



Monitor & Scale

Kubernetes autoscales pods based on traffic while Prometheus and Grafana track latency and errors.

Key Takeaways

Docker = Reproducibility

Package your model, code, and dependencies into an immutable container image that runs the same everywhere — from laptop to cloud.

Kubernetes = Scale

Orchestrate dozens or thousands of containers with automatic scaling, load balancing, self-healing, and zero-downtime deployments.

Helm = Simplicity

Manage complex Kubernetes deployments as versioned, reusable packages — reducing configuration errors and speeding up releases.

Containerization with Docker and orchestration with Kubernetes are not optional extras — they are the **foundational infrastructure** of any production-ready MLOps practice.